CODEWITHZ
DEVELOPER HANDBOOK

THE COMPLETE BACKEND DEVELOPER HANDBOOK

20 Backend Concepts Every Developer Should Know

APIs • DATABASES • SECURITY
PERFORMANCE • SCALABILITY • SYSTEM DESIGN

Practical explanations • Examples • Interview questions • Project blueprint

@CodewithZ

CONTENTS

A practical handbook for college students, freshers, and working developers.

NO.	TOPIC
01	Client–Server Architecture
02	REST APIs
03	HTTP Methods & Status Codes
04	Authentication vs Authorization
05	JWT vs Sessions
06	API Versioning
07	SQL vs NoSQL
08	Database Indexing
09	Transactions & ACID
10	Database Normalization
11	Connection Pooling
12	N+1 Query Problem
13	Caching
14	Load Balancing
15	Horizontal vs Vertical Scaling
16	Message Queues
17	Rate Limiting
18	Retries & Timeouts
19	SQL Injection
20	HTTPS & TLS
BONUS	System flow, interview questions, project, revision checklist

How to use this handbook: For every concept, learn what it is, why it matters, when to use it, and the trade-offs involved.

01. CLIENT–SERVER ARCHITECTURE

Client–Server Architecture is a model in which a client sends a request to a server, and the server processes the request and returns a response. The client may be a browser, mobile app, or another service.

How it works

- The user performs an action in an application.
- The client sends a request to the backend.
- The backend validates and processes the request.
- The backend communicates with a database if needed.
- The server returns a response and the client displays the result.

Example: login flow

- Client submits credentials.
- Backend verifies them against stored credential data.
- The application creates an authenticated session or issues a token.
- The client receives the result and loads the user's permitted content.

Important components

- Client: initiates requests and displays results.
- Server: processes requests and applies business rules.
- Database: stores and retrieves application data.
- Network: carries requests and responses between systems.

Common architectures

- Two-tier: client communicates with a server or data layer.
- Three-tier: presentation, application logic, and data layers are separated.
- Multi-tier: the application is split into additional services or layers.

Interview: When a URL is entered, the browser may resolve DNS, establish a connection, negotiate TLS for HTTPS, send an HTTP request, receive a response, and render the returned resources. The exact sequence varies with caching and connection reuse.

02. REST APIs

REST (Representational State Transfer) is an architectural style for networked applications. A REST API commonly exposes resources through URLs and uses HTTP semantics to retrieve or modify them.

Example endpoints

- GET /api/products — list products.
- GET /api/products/101 — retrieve product 101.
- POST /api/products — create a product.
- PATCH /api/products/101 — update selected fields.
- DELETE /api/products/101 — request removal of a product.

Important REST principles

- Client–server separation: frontend and backend have distinct responsibilities.
- Statelessness: each request carries the context needed to process it; the server should not depend on previous request context.
- Cacheability: responses can specify whether they may be cached.
- Uniform interface: resources are accessed through consistent conventions.
- Layered system: gateways, proxies, and other intermediaries may sit between client and server.
- Code on demand is optional.

Good API practices

- Use nouns for resources and consistent URL conventions.
- Use HTTP methods according to their semantics.
- Return meaningful status codes and consistent error structures.
- Validate inputs and enforce authorization on the server.
- Support pagination for large collections.
- Document compatibility and versioning decisions.

Interview: HTTP is a communication protocol; REST is an architectural style that can use HTTP. Not every HTTP API is RESTful.

03. HTTP METHODS & STATUS CODES

HTTP methods express the intended action on a resource. Status codes communicate the outcome of a request.

Common methods

- GET: retrieve a representation without intending to change server state.
- POST: submit data for processing, commonly to create a resource.
- PUT: generally replace the target resource representation.
- PATCH: apply partial modifications.
- DELETE: request removal of the target resource.

Idempotency

- An operation is idempotent if repeating the same request has the same intended effect on server state as doing it once.
- GET, PUT, and DELETE are defined as idempotent, although response codes may differ.
- POST is not inherently idempotent. Use idempotency keys for operations such as payment creation when duplicate effects must be avoided.

Success codes

- 200 OK — request succeeded.
- 201 Created — resource created.
- 202 Accepted — accepted for processing, which may not be complete.
- 204 No Content — succeeded without a response body.

Client and server errors

- 400 Bad Request — malformed or invalid request.
- 401 Unauthorized — authentication is missing or unsuccessful.
- 403 Forbidden — request is refused, often due to permissions.
- 404 Not Found — resource not found.
- 409 Conflict — conflict with current state.
- 422 Unprocessable Content — semantically invalid content.
- 429 Too Many Requests — rate limit exceeded.
- 500 Internal Server Error — unexpected server failure.
- 502 Bad Gateway — invalid upstream response.
- 503 Service Unavailable — temporarily unavailable.
- 504 Gateway Timeout — upstream did not respond in time.

Interview: 401 generally indicates missing or invalid authentication; 403 means the server refuses access. PUT generally replaces a resource, while PATCH applies partial changes.

04. AUTHENTICATION VS AUTHORIZATION

Authentication verifies identity; authorization decides what an identity may access or do.

Authentication — who are you?

- Email and password login.
- Multi-factor authentication.
- Sign-in through an identity provider.
- API credentials for service-to-service calls.

Authorization — what can you do?

- A user may view their own profile.
- An administrator may manage users.
- A manager may access reports for their team.
- A user may be blocked from modifying another user's records.

Common authorization models

- RBAC (Role-Based Access Control): permissions are assigned to roles.
- ABAC (Attribute-Based Access Control): decisions depend on attributes such as ownership, department, or resource classification.
- ACL (Access Control List): specifies which identities or groups may perform actions on a resource.

Security practices

- Enforce checks on the server, not only in the UI.
- Check resource ownership where needed.
- Follow least privilege and deny by default.
- Test for insecure direct object references, such as changing an ID to access another user's data.

Interview: Successful login does not grant access to every endpoint. Each requested action and resource still needs an authorization check.

05. JWT VS SESSIONS

Sessions and JSON Web Tokens (JWTs) are two ways to carry or maintain authentication state. The right choice depends on architecture, revocation needs, and browser security.

Session flow

- User submits credentials; the server verifies them.
- The server creates a session in server-side storage.
- The client receives a session ID, commonly in a cookie.
- The client sends the cookie with later requests; the server looks up the session.

JWT flow

- Server verifies credentials and issues a token.
- A typical JWT has a header, payload, and signature.
- The client sends the token according to the application's security design.
- The server validates signature and relevant claims such as issuer, audience, and expiration.

Comparison

- Sessions: state is usually server-side; central invalidation is often straightforward; shared session storage may be needed across instances.
- JWTs: claims travel in the token; validation can reduce session lookups in suitable designs; immediate revocation requires additional design.
- A signed JWT is not automatically encrypted; its payload is generally readable by anyone who has it.

Security practices

- Use HTTPS and short-lived access tokens where appropriate.
- Protect refresh tokens and avoid sensitive secrets in token claims.
- Carefully choose cookie or browser storage strategies.
- Consider CSRF risks with cookies and XSS risks when tokens are accessible to JavaScript.

Interview: Neither JWT nor sessions are universally better. Choose based on revocation, scaling, browser threat model, and identity-provider integration.

06. API VERSIONING

API versioning allows a backend to evolve while existing clients continue using a compatible contract.

Common strategies

- URL: /api/v1/users and /api/v2/users.
- Query parameter: /api/users?version=2.
- Header-based versioning: client specifies a version in a request header.
- Media type versioning: version is represented in the Accept media type.

Best practices

- Prefer backward-compatible changes where possible.
- Document changes and migration steps.
- Define a deprecation policy and timeline.
- Monitor older-version usage before retirement.
- A new optional field does not necessarily require a new version.

Interview: Introduce a new version when a change cannot remain backward compatible and clients need a separate contract or migration period.

07. SQL VS NOSQL

SQL databases typically use relational tables and relationships. NoSQL is an umbrella term for several data models, including document, key-value, wide-column, and graph databases.

SQL strengths

- Relational modeling and joins.
- Constraints and transaction support.
- Structured querying and reporting.
- Mature indexing and query optimization.

NoSQL models and examples

- Document: MongoDB.
- Key-value: Redis.
- Wide-column: Cassandra.
- Graph: Neo4j.

Choosing a model

- Use SQL when relational integrity, complex queries, and multi-record transactions are central.
- Consider NoSQL when its data model and distribution characteristics match the workload.
- SQL can scale horizontally; NoSQL is not automatically faster.
- Many systems combine relational storage with caches or specialized data stores.

Interview: An e-commerce application may use a relational database for orders, payments, and inventory integrity, with a cache or NoSQL store for selected workloads.

08. DATABASE INDEXING

An index is a data structure that helps a database locate rows without scanning the entire table for every query.

Example

- Query: SELECT * FROM users WHERE email = 'zaid@example.com';
- A useful index can make the lookup more efficient.
- Example: CREATE INDEX idx_users_email ON users(email);

Common types

- B-tree: common for equality, range, and ordered access.
- Composite: indexes multiple columns; column order matters.
- Unique: enforces uniqueness for indexed key values.
- Full-text: supports text-search workloads in systems that provide it.

Trade-offs and practices

- Indexes use storage and add overhead to inserts, updates, and deletes.
- Index frequently filtered or joined columns when measurements support it.
- Inspect query plans and data distribution.
- Avoid blindly indexing every column.
- The optimizer may choose a table scan when it estimates that to be cheaper.

Interview: Indexes can speed up reads but slow writes because the database must maintain index structures when data changes.

09. TRANSACTIONS & ACID

A transaction groups related database operations into a logical unit. It helps prevent partial updates from leaving data in an invalid state.

Example: bank transfer

- Deduct money from Account A.
- Add the same amount to Account B.
- If a required operation fails, roll back the transaction.
- Production implementations must also validate balances, affected rows, and business rules.

ACID

- Atomicity: all transaction operations succeed or the transaction is rolled back.
- Consistency: defined constraints and business invariants are preserved.
- Isolation: concurrent transactions are controlled according to the isolation level.
- Durability: committed changes are designed to survive failures within the database's guarantees.

Isolation levels and anomalies

- Common levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable.
- Dirty read: reading uncommitted changes.
- Non-repeatable read: rereading a row and seeing a different committed value.
- Phantom read: repeating a query and seeing additional or missing rows due to concurrent changes.
- Exact behavior varies by database system.

Interview: Transactions help coordinate related writes, but distributed payment workflows also need idempotency and careful handling of external services.

10. DATABASE NORMALIZATION

Normalization organizes relational data to reduce duplication and prevent insert, update, and delete anomalies.

Example

- If customer name and email are repeated on every order, changing an email requires multiple updates.
- A CUSTOMERS table can store customer_id, name, and email once.
- An ORDERS table can reference customer_id.

Normal forms

- 1NF: fields contain single values from their intended domains; repeating groups are avoided.
- 2NF: in 1NF, with every non-key attribute depending on the whole candidate key rather than part of a composite key.
- 3NF: in 2NF, with no prohibited transitive dependency of non-key attributes on a candidate key.

Denormalization

- Intentionally introduces duplication or precomputed data to improve reads or simplify access patterns.
- Can help analytics and high-read workloads.
- Adds consistency, refresh, and maintenance complexity.

Interview: Normalization improves integrity and maintainability; selective denormalization may be useful when measurement shows a performance need.

11. CONNECTION POOLING

Connection pooling reuses database connections instead of opening a new connection for every request.

How it works

- Application requests a connection from the pool.
- The pool provides an available connection.
- The application executes database operations.
- The connection is returned to the pool for reuse.

Benefits

- Reduces connection-establishment overhead.
- Can improve throughput under suitable workloads.
- Limits concurrent connections and helps manage database resources.

Configuration and mistakes

- Maximum pool size limits connections in a pool.
- Timeouts control how long acquisition or connection attempts wait, depending on the driver.
- Avoid oversized pools that overwhelm the database.
- Dispose of connections properly and do not hold them during unrelated slow work.
- Many application instances can collectively create too many connections.

Interview: Pooling does not mean every connection stays open forever; the pool manages reusable connections according to its configuration and workload.

12. THE N+1 QUERY PROBLEM

The N+1 problem occurs when an application fetches a collection with one query, then issues another query for each item.

Example

- One query fetches 100 orders.
- A separate query fetches the customer for each order.
- The total can be approximately 101 queries.

Why it hurts

- More network round trips and latency.
- Increased database load.
- Performance degrades as collection size grows.

Solutions

- Use a JOIN when a combined result is appropriate.
- Use ORM eager loading or batch fetching where suitable.
- Batch repeated lookups with DataLoader-style approaches in appropriate systems.
- Inspect SQL logs, query counts, and execution plans.
- Avoid joining multiple large one-to-many relationships blindly, as row counts can multiply.

Interview: Debug by measuring query counts and timings, identifying repeated lookups, then comparing joins, batching, and eager-loading strategies.

13. CACHING

Caching stores frequently accessed data in a faster layer so the system can avoid repeating expensive work.

Common cache types

- Browser cache: stores eligible resources in the browser.
- CDN cache: stores content at edge locations.
- Application cache: stores computed or frequently used values.
- Distributed cache: a shared system such as Redis or Memcached.
- Database systems may also cache internal data or selected results.

Cache-aside pattern

- Check the cache first.
- On a hit, return the cached value.
- On a miss, query the source of truth.
- Store the result in cache, then return it.

Common problems

- Cache invalidation: remove or refresh old values after source data changes.
- Cache stampede: many requests reload the same expired value simultaneously.
- Stale data: cache contains an outdated value.
- Cache penetration: repeated misses bypass the cache.

Practices

- Choose appropriate TTLs.
- Invalidate or update entries when data changes.
- Consider negative caching, request coalescing, or locks for expensive misses.
- Protect sensitive data and monitor hit rate, latency, memory, and evictions.

Interview: Caching improves suitable read workloads, but introduces freshness and invalidation trade-offs.

14. LOAD BALANCING

A load balancer distributes incoming requests across multiple server instances to reduce bottlenecks and support availability.

Common algorithms

- Round robin: send requests in sequence across eligible servers.
- Weighted round robin: assign more traffic to servers with higher weights.
- Least connections: prefer fewer active connections, depending on implementation.
- IP hash: select a backend based on client IP information.

Health checks and benefits

- Health checks help remove unhealthy instances from rotation.
- Traffic can be distributed across instances.
- Rolling maintenance and deployments can be supported with sufficient healthy capacity.
- Redundancy can improve availability.

Design considerations

- Avoid relying on a single server for session state.
- Use meaningful readiness checks.
- Database and other dependencies must also have enough capacity.
- A load balancer itself may need redundancy.
- Retries can amplify overload if poorly configured.

Interview: When an instance fails health checks, the load balancer can stop routing new requests to it, assuming other instances have sufficient capacity.

15. HORIZONTAL VS VERTICAL SCALING

Scalability is a system's ability to handle more workload while meeting performance and reliability requirements.

Vertical scaling

- Increase CPU, RAM, or storage resources on an existing machine.
- Often simpler initially.
- Has machine and service limits; a single machine can remain a failure point.

Horizontal scaling

- Add more machines or application instances.
- Can increase capacity and resilience when designed correctly.
- Requires attention to distributed state, network failures, coordination, and shared dependencies.

Comparison and example

- A small dashboard may work well on one larger server.
- A global commerce system may need multiple app instances, caching, replicas, and data partitioning.
- Choose based on cost, workload, performance, availability, and operational complexity.

Interview: Horizontal scaling is not always better; it can improve capacity but introduces distributed-system complexity.

16. MESSAGE QUEUES

A message queue or event-streaming platform lets one component publish work or events for another component to process asynchronously.

Example: order processing

- Order API validates and records an order.
- A message is published for downstream processing.
- A consumer sends confirmation email.
- Other consumers may update analytics or notify a warehouse.

Common technologies

- RabbitMQ: message broker with queueing and routing features.
- Apache Kafka: distributed event-streaming platform with durable streams and consumer groups.
- Amazon SQS: managed queue service.
- Delivery, ordering, retention, and operations differ by technology.

Important concepts

- Producer publishes messages; consumer processes them; broker distributes them.
- Acknowledgment records processing progress according to the protocol.
- Dead-letter queue stores messages that cannot be processed under the configured policy.
- Idempotent consumers handle duplicate delivery without repeating unintended effects.

Delivery semantics

- At-most-once: a message may be lost, but is not redelivered under the guarantee.
- At-least-once: a message may be delivered more than once.

- Exactly-once behavior is limited to defined system boundaries and does not automatically guarantee exactly-once effects in every external system.

Interview: Queues buffer work and reduce coupling, but reliable processing still requires persistence, acknowledgments, retries, idempotency, monitoring, and dead-letter handling.

17. RATE LIMITING

Rate limiting restricts how frequently a client or identity can make requests within a defined policy.

Example

- An API might allow 100 requests per minute per authenticated user.
- When the limit is exceeded, return 429 Too Many Requests.
- A Retry-After header may communicate when to retry.

Common algorithms

- Fixed window: counts requests in fixed intervals; simple but can allow boundary spikes.
- Sliding-window log: tracks timestamps in a rolling window; accurate but potentially expensive.
- Sliding-window counter: approximates a rolling window using adjacent interval counters.
- Token bucket: requests consume tokens; supports controlled bursts.
- Leaky bucket: controls the rate at which requests are processed, depending on implementation.

Where and how

- Apply at an API gateway, proxy, middleware, or endpoint.
- Use different policies for login, expensive operations, and general reads.
- Distributed applications may need shared counters or atomic operations.
- IP-only policies can affect users behind shared networks.
- Monitor rejected requests and tune limits based on real traffic.

Interview: A distributed rate limiter can use an API gateway or shared store such as Redis with atomic counter operations or a suitable script.

18. RETRIES & TIMEOUTS

Timeouts bound how long an operation can wait. Retries repeat an operation after failures that may be temporary.

Timeout examples

- Connection timeout.
- HTTP request timeout.
- Database command timeout.
- Upstream dependency timeout.

Retry strategy

- Retry only failures likely to be transient.
- Bound the number of attempts.

- Use exponential backoff and random jitter.
- Respect service guidance and avoid retrying indefinitely.
- Use idempotency protection for operations that must not create duplicate effects.

Circuit breaker

- Temporarily blocks calls to a dependency after a failure threshold.
- May allow limited trial requests later to check whether the dependency recovered.
- Can help prevent repeated calls from overwhelming a failing service.

Interview: Retries can worsen an outage by increasing load. Bounded retries, backoff, jitter, timeouts, and circuit breakers help control the failure.

19. SQL INJECTION

SQL injection occurs when untrusted input changes the intended structure or meaning of a SQL query, often because user input is concatenated into query strings.

Safe approach: parameterized queries

- Use query parameters so input is treated as data, not SQL syntax.
- In C#, use SqlCommand with named parameters.
- In Java, use PreparedStatement with ? placeholders.
- Choose parameter types and sizes that match the database schema where possible.

Additional protections

- Validate input according to business rules.
- Use least-privilege database accounts.
- Avoid exposing raw database errors to users.
- Keep drivers and frameworks updated.
- Use safe ORM APIs and avoid unsafe raw SQL construction.
- Validation alone is not a replacement for parameterization.

Interview: Prepared statements separate query structure from values, preventing input from being interpreted as part of the SQL statement.

20. HTTPS & TLS

HTTPS is HTTP protected by Transport Layer Security (TLS). It helps protect data in transit from interception and tampering.

What TLS provides

- Confidentiality: encrypts communication.
- Integrity: helps detect unauthorized modification.
- Authentication: certificates and the handshake help authenticate the server; client authentication is also possible in some configurations.

Simplified flow

- Client connects to the server.
- Server presents a certificate.
- Client validates certificate and hostname.
- The parties negotiate cryptographic parameters and establish session keys.
- HTTP data is exchanged through the protected connection.

Best practices

- Use valid certificates and current TLS configurations.
- Redirect HTTP to HTTPS where appropriate.
- Use Secure cookies for sensitive browser sessions.
- Consider HSTS and avoid mixed content.
- Continue to implement authorization, validation, and secure business logic.

Interview: HTTPS protects data in transit, but does not automatically prevent broken authorization, SQL injection, insecure business logic, or exposed secrets.

BONUS 1. HOW THE 20 CONCEPTS WORK TOGETHER

Example: a user purchasing a product from an e-commerce application.

1. Request

Frontend sends an HTTP request to a REST API over HTTPS. Concepts: client–server architecture, REST, HTTP methods, TLS.

2. Authentication

Backend verifies identity and permissions. Concepts: authentication, authorization, JWT or sessions.

3. Data retrieval

Backend reads product and inventory data. Concepts: SQL/NoSQL, indexing, normalization, connection pooling.

4. Performance

Cache serves frequently accessed product information; queries are optimized to avoid unnecessary round trips. Concepts: caching, N+1.

5. Order creation

Backend validates inventory and records the order transactionally. Concept: transactions and ACID.

6. Traffic management

A load balancer routes requests across healthy instances. Concepts: load balancing and horizontal scaling.

7. Background work

A queue processes confirmation emails and analytics asynchronously. Concept: message queues.

8. Protection

Rate limiting controls excessive traffic and parameterized queries protect database access. Concepts: rate limiting and SQL injection prevention.

9. Failure handling

Timeouts, bounded retries, and idempotency help prevent duplicate effects. Concept: retries and timeouts.

10. Compatibility

API versioning supports older clients while the backend evolves. Concept: API versioning.

Key takeaway: backend engineering means building systems that are secure, reliable, maintainable, and capable of handling real-world traffic.

BONUS 2. 15 IMPORTANT BACKEND INTERVIEW QUESTIONS

1. What happens when you enter a URL in a browser?

Discuss DNS, connection establishment, TLS, HTTP requests, server processing, and browser rendering.

2. What is the difference between REST and GraphQL?

REST commonly exposes resources through endpoints; GraphQL lets clients request specific fields through a schema-based query language.

3. What is the difference between PUT and PATCH?

PUT generally replaces a resource representation; PATCH applies partial modifications.

4. Authentication vs authorization?

Authentication verifies identity; authorization checks permissions.

5. JWT vs sessions — which would you choose?

Discuss state management, revocation, scaling, storage, and browser security.

6. SQL vs NoSQL — when would you use each?

Discuss relationships, consistency, access patterns, query needs, and scaling requirements.

7. How does an index improve performance?

It can reduce data examined for supported queries, at the cost of storage and write overhead.

8. Explain ACID.

Atomicity, Consistency, Isolation, and Durability.

9. What is the N+1 query problem?

One query retrieves a collection, followed by one additional query per item.

10. Caching vs indexing?

Caching reuses stored values or results; indexing helps locate data efficiently in a data store.

11. How would you scale an API handling millions of requests?

Discuss profiling, caching, load balancing, stateless instances, database optimization, asynchronous work, and monitoring.

12. Why use message queues?

To decouple producers and consumers, buffer work, and process suitable tasks asynchronously.

13. How do you prevent SQL injection?

Use parameterized queries, safe database APIs, least-privilege access, and appropriate validation.

14. How would you handle a downstream service outage?

Discuss timeouts, bounded retries, exponential backoff, jitter, circuit breakers, safe fallbacks, and observability.

15. How would you prevent duplicate payment requests?

Use idempotency keys, persist operation state, enforce suitable uniqueness constraints, and understand the provider's guarantees.

BONUS 3. PRACTICAL PROJECT — URL SHORTENER

Build a service that converts long URLs into short links and redirects users to the original destinations.

Core features

- User registration and login.
- Create short URLs and redirect short links.
- Track click counts and view analytics.
- Rate-limit URL creation.
- Cache frequently accessed URL mappings.

Suggested database tables

TABLE	FIELDS
USERS	id, name, email, password_hash, created_at
URLS	id, user_id, short_code, original_url, created_at, expires_at
URL_CLICKS	id, url_id, clicked_at, referrer

Example API endpoints

```
POST /api/auth/register
POST /api/auth/login
POST /api/urls
GET /api/urls/{shortCode}
GET /api/urls/{shortCode}/analytics
DELETE /api/urls/{id}
```

Concepts you will practice

- REST API design and HTTP status codes.
- Authentication, authorization, and database schema design.
- Indexing short_code and enforcing uniqueness constraints.
- Transactions, caching with Redis, and rate limiting.
- Input validation and safe URL handling.
- Connection pooling, API versioning, horizontal scaling, monitoring, and failure handling.

Advanced challenges

1. Prevent duplicate short codes with a database uniqueness constraint.
2. Cache popular URL mappings.
3. Design a reliable click-tracking pipeline.
4. Use a queue for analytics processing.
5. Protect against malicious URLs and abuse.
6. Measure API latency and database query performance.
7. Explain how the service would handle millions of redirects.

Interview tip: Be ready to explain why you chose each design, what trade-offs it introduces, and how you would measure whether it works.

FINAL REVISION CHECKLIST

Use this list to identify concepts you can explain clearly and those that need another review.

- ■ Client–Server Architecture
- ■ REST APIs and HTTP Methods
- ■ HTTP Status Codes
- ■ Authentication vs Authorization
- ■ JWT vs Sessions
- ■ API Versioning
- ■ SQL vs NoSQL
- ■ Database Indexing
- ■ Transactions and ACID
- ■ Database Normalization
- ■ Connection Pooling
- ■ N+1 Query Problem
- ■ Caching and Cache Invalidation
- ■ Load Balancing
- ■ Horizontal vs Vertical Scaling
- ■ Message Queues
- ■ Rate Limiting
- ■ Retries, Timeouts, and Circuit Breakers
- ■ SQL Injection Prevention
- ■ HTTPS and TLS

Final advice

Do not memorize definitions alone. For each concept, learn three things:

1. What is it? Understand the core idea.
2. Why do we need it? Identify the problem it solves.
3. When would you use it? Explain the trade-offs using a real project.

If you can explain these concepts clearly and connect them to a practical project, you will have a strong foundation for backend development interviews.

CREATED FOR THE CODEWITHZ DEVELOPER COMMUNITY

Follow @CodewithZ for backend development, system design, coding, and interview preparation resources.